Project Initialization and Planning Phase

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Project Proposal (Proposed Solution) report

OptiCrop is a local Python-based web application that uses data-driven machine learning to optimize agricultural production. It integrates soil nutrient levels (N, P, K), soil pH, temperature, humidity, and rainfall to provide intelligent crop recommendations, crop suitability assessments, fertilizer guidance, plant disease identification, real-time weather data, and an interactive research analytics dashboard — all in a single no-login, no-framework, offline-capable tool.

Project Overview

Objective	Develop a Python HTTP server web application that recommends the optimal crop for given soil and climate conditions, identifies plant diseases from symptoms and leaf images, provides fertilizer correction advice, fetches real-time weather data, and enables agricultural researchers to explore crop-environment patterns through an interactive analytics dashboard.
Scope	OptiCrop addresses three scenarios: (1) Smart Crop Recommendation and Suitability Assessment for farmers entering N, P, K, temperature, humidity, pH, and rainfall; (2) Plant Disease Identification via symptom selection or leaf image upload; and (3) Agricultural Research and Policy Planning via an analytics dashboard with filterable crop-environment data and CSV export. The app runs locally at http://127.0.0.1:8000 with no cloud dependency.
Problem Statement Description	Smallholder farmers in India lack affordable, accessible tools for data-driven crop decisions. Soil testing costs ■500–■2000 per test. Plant diseases go undetected until visible crop damage. Fertilizer is over- or under-applied due to lack of guidance. Existing tools are fragmented, require logins, or depend on paid APIs — creating a gap that OptiCrop fills.
Impact	OptiCrop eliminates guesswork in crop selection, enables early disease detection, and provides targeted fertilizer advice — reducing input waste and improving yield outcomes. The analytics dashboard supports evidence-based policy planning for agricultural researchers and government bodies.
Approach	1. Train a multi-class crop classifier using an ensemble of Random Forest (n=350), Decision Tree, and SVC models on Crop_recommendation.csv (2200 rows, 22 crops). 2. Train a Random Forest disease classifier on opticrop-data.json disease profiles with data augmentation. 3. Optionally train a CNN (MobileNetV2 base) for image-based disease identification on the PlantVillage dataset. 4. Build a Python ThreadingHTTPServer to serve static files and 7 REST API endpoints. 5. Develop 5 responsive HTML pages with Vanilla JS frontend and Chart.js analytics dashboard. 6. Integrate Open-Meteo API for free, key-free real-time weather auto-fill.

Key Features

#	Feature	Description	Status
1	Crop Recommendation	POST /api/recommend — RF+DT+SVC ensemble predicts best crop from 9 features	Done

2	Crop Suitability Assessment	POST /api/suitability — weighted score across 7 metrics with remediation steps	Done
3	Fertilizer Recommendation	POST /api/fertilizer — NPK-gap analysis with crop-specific nutrient advice	Done
4	Symptom Disease Detection	POST /api/disease — RF classifier on crop + symptoms + temperature/humidity/pH	Done
5	Image Disease Detection	POST /api/image-disease — CNN or pixel-feature RF on uploaded leaf photo	Done
6	Real-time Weather Auto-fill	GET /api/weather — Open-Meteo API, no key, city name or GPS coordinates	Done
7	Research Analytics Dashboard	GET /api/analytics — Chart.js bar/radar/scatter, filter by crop/region/season, CSV export	Done

Resource Requirements

Computing Resources	Local CPU — any modern laptop/desktop (Intel i5 or equivalent). No GPU required for inference. Google Colab T4 GPU used only for optional CNN training.
Memory	4 GB RAM minimum for local inference. 8 GB RAM recommended when loading CNN model alongside the server.
Storage	~50 MB for trained model files (.joblib). ~200 KB for Crop_recommendation.csv. ~5 GB for optional PlantVillage image dataset (only needed for CNN training, not for inference).
Frameworks	None required for server — Python built-in http.server. Scikit-learn for ML. TensorFlow-CPU (optional) for CNN inference.
Libraries	pandas>=2.2.0, numpy>=1.9.0, scikit-learn>=1.4.0, matplotlib>=3.8.0, seaborn>=0.13.0, joblib>=1.4.0, Pillow>=10.0.0, tensorflow-cpu>=2.14.0 (optional)
Development Environment	VS Code or any text editor. Python 3.10+. pip for dependency installation. Git for version control.
Data	Kaggle Crop Recommendation Dataset (2200 rows, 8 columns, 22 crop classes). PlantVillage Plant Disease Dataset (optional, for CNN training). Custom disease profiles in opticrop-data.json.